



University of Computer Studies, Yangon (UCSY)

Classification: Basic Concepts and Methods

IS-212
(Data and Knowledge Mining)

Dr. Hsu Myat Mo
Faculty of Information Science, UCSY



References

- Data Mining Concepts and Techniques, Jiawei Han, Micheline Kamber, Jian Pei. 3rd Edition, Morgan Kaufmann.
- Introduction to Data Mining, Pang-Ning Tan, Michael Steinbach, Vipin Kumar, Addison Wesley, 2006.
- Data Mining – Practical Machine Learning Tools and Techniques (2nd Ed.), Ian H. Witten and Eibe Frank, Morgan Kaufmann, 2005.
- Predictive Data Mining, S.M. Weiss and N. Indurkha, Morgan Kaufmann, 1998.
- Data Mining – Introductory and Advanced Topics, Margaret H. Dunham, Prentice Hall, 2003.
- Data Mining with R – Learning with Case Studies, Torgo, Luís, CRC Press, Taylor and Francis Group, LLC, 2011



Course Objectives

- To explain the workflow of data mining techniques
- To remember how to manipulate data mining techniques with Python and R-Programming
- To explain how to evaluate the experimental results of data mining techniques
- To express the visualization of the experimental results with Python and R-Programming



Outlines

- Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods
- Rule Based Classification Methods
- Model Evaluation and Selection
- Techniques to Improve Classification Accuracy
- Summary



Introduction

- **Classification** is a form of data analysis that extracts models **describing important data classes**.
- Such models, called classifiers, predict categorical (discrete, unordered) class labels.
- Data analysis task of numeric prediction, where the model constructed predicts a continuous-valued function, or ordered value, as opposed to a class label.
- This model is a predictor.
- **Regression** analysis is a statistical methodology that is most often used **for numeric prediction**.
- Classification and numeric prediction are the two major types of prediction problems.
- **Ranking** is another type of **numerical prediction** where the model predicts the ordered values (i.e., ranks).



General Approach to Classification

- “How does classification work?”
- Data classification is a two-step process:
 - a learning step (where a classification model is constructed)
 - a classification step (where the model is used to predict class labels for given data)
- This first step of the classification process can also be viewed as the learning of a mapping or function, $y = f(X)$, that can predict the associated class label y of a given tuple X .
- learn a mapping or function that separates the data classes.
- The learning of the classifier is “supervised” in that it is told to which class each training tuple belongs).
 - Learning by examples



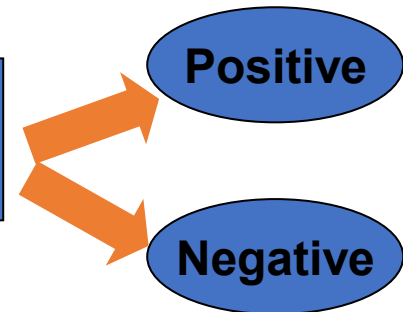
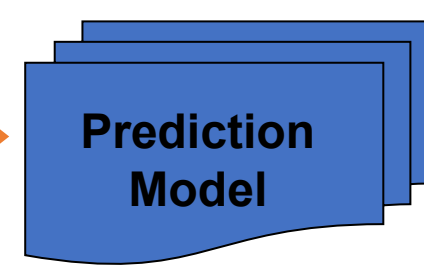
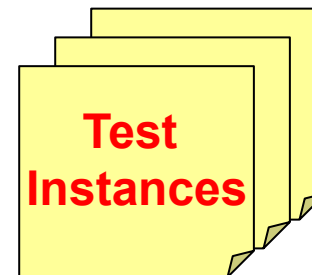
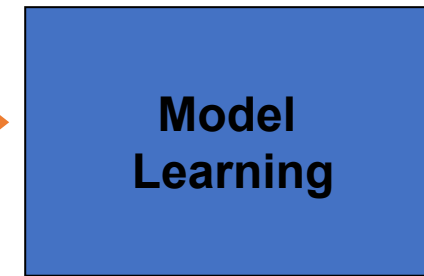
Cont'd

➤ Supervised learning (classification)

- Supervision: The training data such as observations or measurements are accompanied by **labels** indicating the classes which they belong to
- New data is classified based on the models built from the training set

Training Data with class label:

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No





Cont'd

- “What about classification accuracy?”
 - The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier.
 - The associated class label of each test tuple is compared with the learned classifier's class prediction for that tuple.



Model Evaluation and Selection

➤ Metrics for Evaluating Classifier Performance

True positives (TP) : These refer to the positive tuples that were correctly labeled by the classifier. Let TP be the number of true positives.

True negatives (TN) : These are the negative tuples that were correctly labeled by the classifier. Let TN be the number of true negatives.

False positives (FP) : These are the negative tuples that were incorrectly labeled as positive (e.g., tuples of class *buys computer* D *no* for which the classifier predicted *buys computer* D *yes*). Let FP be the number of false positives.

False negatives (FN) : These are the positive tuples that were mislabeled as negative (e.g., tuples of class *buys computer* D *yes* for which the classifier predicted *buys computer* D *no*). Let FN be the number of false negatives.

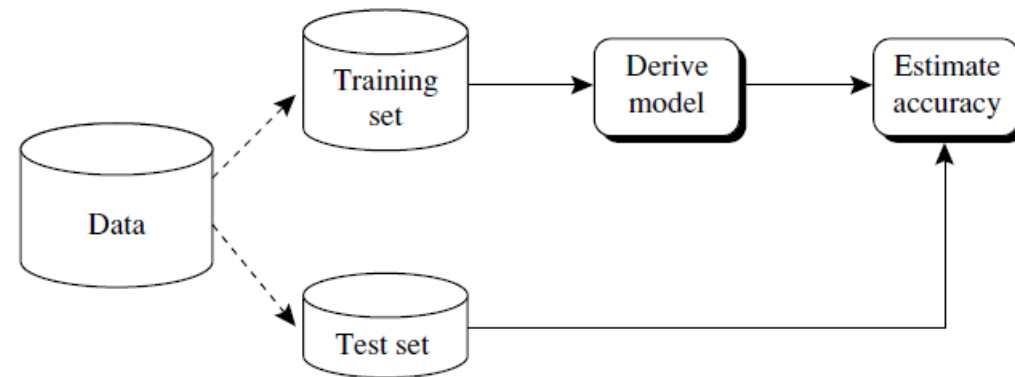
Measure	Formula
accuracy, recognition rate	$\frac{TP + TN}{P + N}$
error rate, misclassification rate	$\frac{FP + FN}{P + N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP + FP}$
F , F_1 , F -score, harmonic mean of precision and recall	$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
F_β , where β is a non-negative real number	$\frac{(1 + \beta^2) \times \text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$

		Predicted class		
		yes	no	
Actual class	yes	TP	FN	P
	no	FP	TN	N
Total		P'	N'	$P + N$

Model Evaluation and Selection



- Holdout Method
- Cross-Validation
 - k -fold cross-validation



What makes a pattern interesting?



- Several **objective measures of pattern interestingness** exist.
- Although objective measures help identify interesting patterns, they are often insufficient unless combined with subjective measures that reflect a particular user's needs and interests.
- **Subjective interestingness measures** are based on user beliefs in the data.

Data Mining Function: Classification



- Typical methods
 - Decision trees, naïve Bayesian classification, support vector machines, KNN, neural networks, rule-based classification, pattern-based classification, logistic regression, ...
- Typical applications:
 - Credit card fraud detection, direct marketing, classifying stars, diseases, web-pages, ...

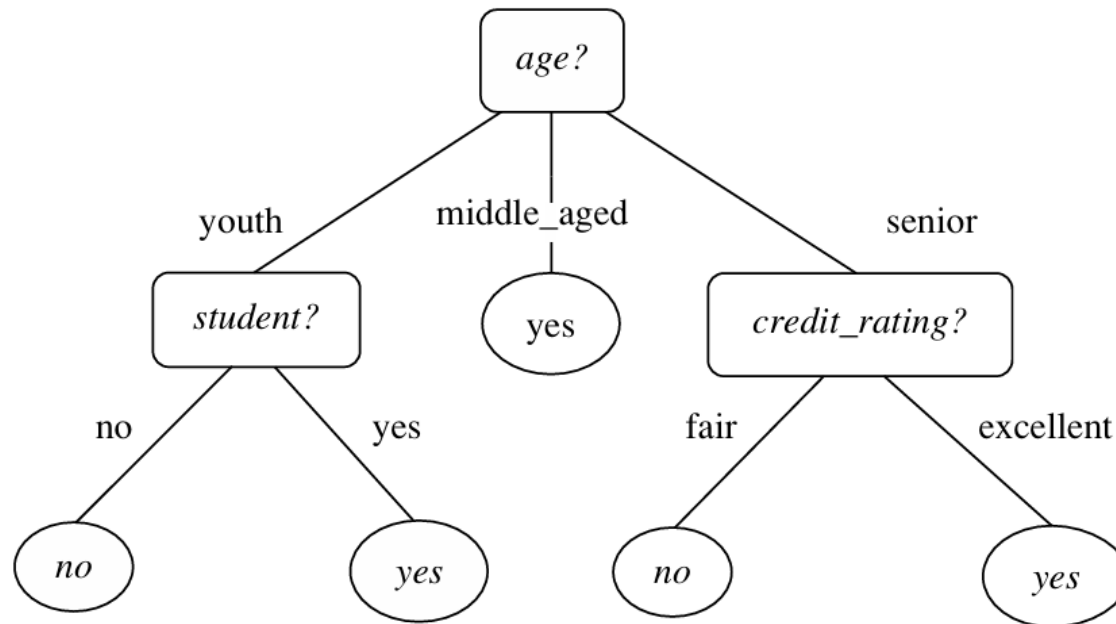


Decision Tree Induction

- Decision tree induction is the learning of decision trees from class-labeled training tuples.
- A decision tree is a flowchart-like tree structure, where each internal node (non leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (or terminal node) holds a class label.
- The topmost node in a tree is the root node.



Decision Tree Induction





Cont'd

- “How are decision trees used for classification?”
 - Given a tuple, X , for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree.
 - A path is traced from the root to a leaf node, which holds the class prediction for that tuple.
 - Decision trees can easily be converted to classification rules.
- During tree construction, attribute selection measures are used to select the attribute that best partitions the tuples into distinct classes.
- When decision trees are built, many of the branches may reflect noise or outliers in the training data. Tree pruning attempts to identify and remove such branches, with the goal of improving classification accuracy on unseen data.



Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition, D .

Input:

- Data partition, D , which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that “best” partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split-point* or *splitting_subset*.

Output: A decision tree.

Method:

- (1) create a node N ;
- (2) **if** tuples in D are all of the same class, C , **then**
- (3) return N as a leaf node labeled with the class C ;
- (4) **if** *attribute_list* is empty **then**
- (5) return N as a leaf node labeled with the majority class in D ; // majority voting
- (6) apply **Attribute_selection_method**(D , *attribute_list*) to **find** the “best” *splitting_criterion*;
- (7) label node N with *splitting_criterion*;
- (8) **if** *splitting_attribute* is discrete-valued **and**
 multiway splits allowed **then** // not restricted to binary trees
- (9) *attribute_list* $\leftarrow$ *attribute_list* – *splitting_attribute*; // remove *splitting_attribute*
- (10) **for each** outcome j of *splitting_criterion*
 // partition the tuples and grow subtrees for each partition
- (11) let D_j be the set of data tuples in D satisfying outcome j ; // a partition
- (12) **if** D_j is empty **then**
- (13) attach a leaf labeled with the majority class in D to node N ;
- (14) **else** attach the node returned by **Generate_decision_tree**(D_j , *attribute_list*) to node N ;
- endfor**
- (15) return N ;



Splitting Criterion

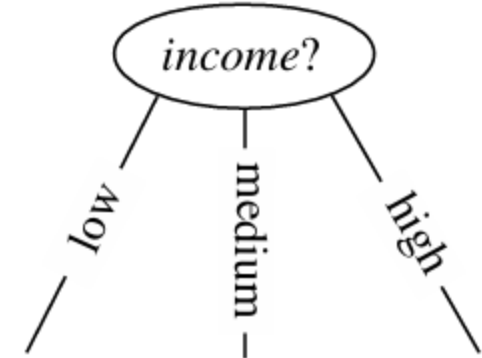
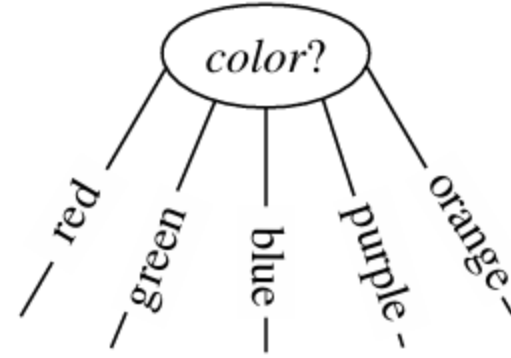
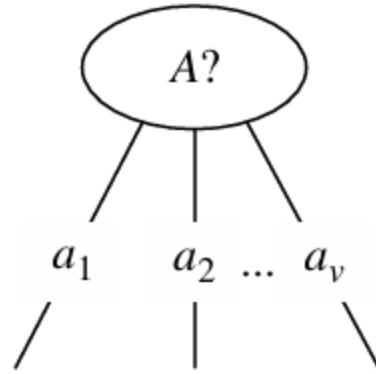
- The splitting criterion tells us which attribute to test at node N by determining the “best” way to separate or partition the tuples in D into individual classes.
- The splitting criterion indicates the splitting attribute and may also indicate either a split-point or a splitting subset.
- The splitting criterion is determined so that, ideally, the resulting partitions at each branch are as “pure” as possible.



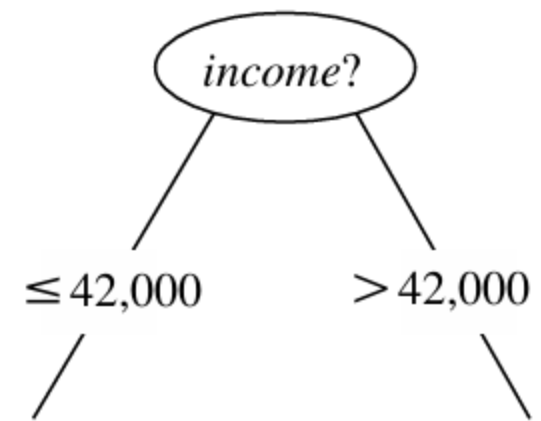
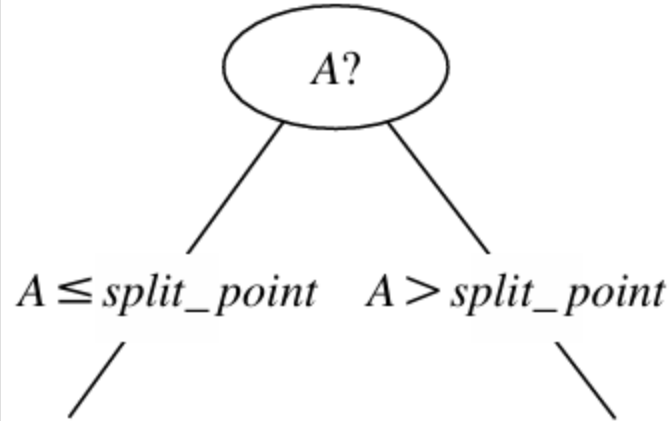
Partitioning scenarios

Examples

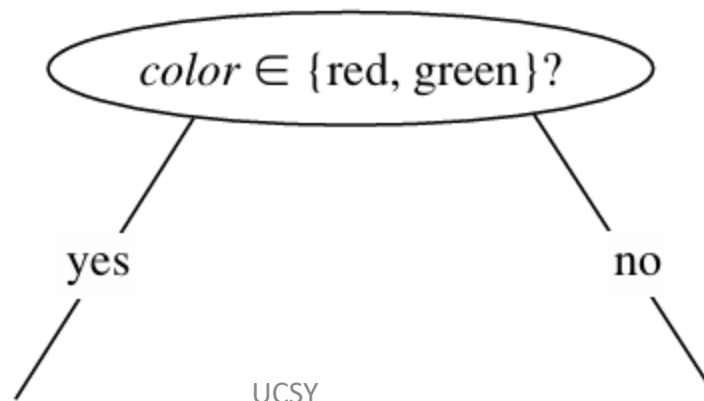
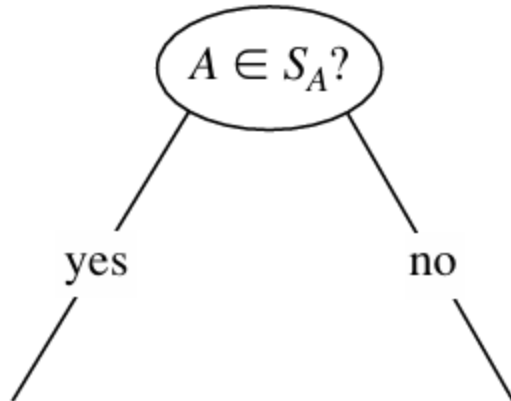
(a)



(b)



(c)





Attribute Selection Measures

- The attribute selection measure provides a ranking for each attribute describing the given training tuples.
- The attribute having the best score for the measure is chosen as the splitting attribute for the given tuples.
- If the splitting attribute is continuous-valued or if we are restricted to binary trees, then, respectively, either a split point or a splitting subset must also be determined as part of the splitting criterion.
- Three popular attribute selection measures—information gain, gain ratio, and Gini index.



Attribute Selection Measures

Class-Labeled Training Tuples from the *AlIElectronics* Customer Database

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no



Information Gain

- This measure is based on information theory
- Let node N represent or hold the tuples of partition D . The attribute with the highest information gain is chosen as the splitting attribute for node N .
- This attribute minimizes the information needed to classify the tuples.
- Such an approach minimizes the expected number of tests needed to classify a given tuple
- The expected information needed to classify a tuple in D is given by $Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$
- How much more information would we still need $Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$
- Information gain is defined as the difference between the original information requirement and the new requirement. $Gain(A) = Info(D) - Info_A(D)$.



Cont'd

- to compute the expected information needed to classify a tuple in D :

$$Info(D) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.940 \text{ bits.}$$

- the expected information needed to classify a tuple in D if the tuples are partitioned according to age is

$$\begin{aligned} Info_{age}(D) &= \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) \\ &\quad + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) \\ &\quad + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) \\ &= 0.694 \text{ bits.} \end{aligned}$$

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$



Gain Ratio

- The information gain measure is biased toward tests with many outcomes.
- Gain ratio attempts to overcome this bias.
- It applies a kind of normalization to information gain using a “split information” value defined analogously with $\text{Info}(D)$ as

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

- This value represents the potential information generated by splitting the training data set, D , into v partitions, corresponding to the v outcomes of a test on attribute A .
- The gain ratio is defined as
$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}$$
- The attribute with the maximum gain ratio is selected as the splitting attribute.



Cont'd

- Computation of gain ratio for the attribute income.

$$\begin{aligned} SplitInfo_{income}(D) &= -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) \\ &= 1.557. \end{aligned}$$

- Gain(income) = 0.029
- GainRatio(income) = 0.029/1.557 = 0.019



Gini Index

- Gini index measures the impurity of D , a data partition or set of training tuples, as

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2,$$

- where p_i is the probability that a tuple in D belongs to class C_i and is estimated by $|C_i, D|/|D|$.
- The sum is computed over m classes.
- The Gini index considers a binary split for each attribute.
- If a binary split on A partitions D into D_1 and D_2 , the Gini index of D given that partitioning is

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

- The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$



Induction of a decision tree using the Gini index

➤ Gini index to compute the impurity of D:

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

➤ Consider the subset {low, medium}

$$Gini_{income \in \{low, medium\}}(D)$$

$$= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2)$$

$$= \frac{10}{14} \left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \frac{4}{14} \left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right)$$

$$= 0.443$$

$$= Gini_{income \in \{high\}}(D).$$



Cont'd

- Gini index values for splits on the remaining subsets are 0.458 (for the sub sets {low, high} and {medium}) and 0.450 (for the subsets {medium, high} and {low}).
- The best binary split for attribute income is on {low, medium} (or {high}) because it minimizes the Gini index.
- Evaluating age, {youth, senior} (or {middle aged}) as the best split for age with a Gini index of 0.375; the attributes student and credit rating are both binary, with Gini index values of 0.367 and 0.429, respectively
- The attribute age and splitting subset {youth, senior} therefore give the minimum Gini index overall, with a reduction in impurity of $0.459 - 0.357 = 0.102$.
- The binary split “age $\in$ {youth, senior?}” results in the maximum reduction in impurity of the tuples in D and is returned as the splitting criterion.



Bayes Classification Methods

- Bayesian classifiers are statistical classifiers.
- They can predict class **membership probabilities** such as the probability that a given tuple belongs to a particular class.
- Determine $P(H|X)$, the probability that the hypothesis H holds given the “evidence” or observed data tuple X . In other words, looking for the probability that tuple X belongs to class C , given that we know the attribute description of X
- Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called class **conditional independence**.

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$



Cont'd

- “How are these probabilities estimated?”
- Suppose that there are m classes, $C_1, C_2, \dots, C_m$. Given a tuple, X , the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X . That is, the Naïve Bayesian classifier predicts that tuple X belongs to the class C_i if and only if

$$P(C_i|X) > P(C_j|X) \quad \text{for } 1 \leq j \leq m, j \neq i.$$

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}$$

- The class C_i for which $P(C_i|X)$ is maximized is called the maximum posteriori hypothesis.



Cont'd

- To simplify the task, naïve Bayesian classifiers assume attributes have **independent distributions**, and thereby estimate

$$\begin{aligned} P(\mathbf{X}|C_i) &= \prod_{k=1}^n P(x_k|C_i) \\ &= P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i) \end{aligned}$$



Predicting a class label using naïve Bayesian classification

- The same training data in slide 19.
- The class label attribute, buys computer, has two distinct values (namely, {yes, no}).
- Let C_1 correspond to the class buys_computer = yes and C_2 correspond to buys_computer = no.
- The tuple we wish to classify is
- $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit rating} = \text{fair})$
- Need to maximize $P(X | C_i)P(C_i)$, for $i = 1, 2$.
- $P(C_i)$, the prior probability of each class,
 - $P(\text{buys_computer} = \text{yes}) = 9/14 = 0.643$
 - $P(\text{buys_computer} = \text{no}) = 5/14 = 0.357$



Cont'd

- To compute $P(X | C_i)$, for $i = 1, 2$, compute the following conditional probabilities:
- $P(\text{age} = \text{youth} \mid \text{buys computer} = \text{yes}) = 2/9 = 0.222$
- $P(\text{age} = \text{youth} \mid \text{buys computer} = \text{no}) = 3/5 = 0.600$
- $P(\text{income} = \text{medium} \mid \text{buys computer} = \text{yes}) = 4/9 = 0.444$
- $P(\text{income} = \text{medium} \mid \text{buys computer} = \text{no}) = 2/5 = 0.400$
- $P(\text{student} = \text{yes} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$
- $P(\text{student} = \text{yes} \mid \text{buys computer} = \text{no}) = 1/5 = 0.200$
- $P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$
- $P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{no}) = 2/5 = 0.400$



Cont'd

- $P(X | \text{buys_computer} = \text{yes}) = P(\text{age} = \text{youth} | \text{buys_computer} = \text{yes}) \times P(\text{income} = \text{medium} | \text{buys_computer} = \text{yes}) \times P(\text{student} = \text{yes} | \text{buys_computer} = \text{yes}) \times P(\text{credit rating} = \text{fair} | \text{buys_computer} = \text{yes})$
 $= 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044.$
- $P(X | \text{buys_computer} = \text{no}) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019.$
- To find the class, C_i , that maximizes $P(X | C_i)P(C_i)$, compute
 - $P(X | \text{buys_computer} = \text{yes})P(\text{buys_computer} = \text{yes}) = 0.044 \times 0.643 = 0.028$
 - $P(X | \text{buys_computer} = \text{no})P(\text{buys_computer} = \text{no}) = 0.019 \times 0.357 = 0.007$
- Therefore, the naïve Bayesian classifier predicts $\text{buys_computer} = \text{yes}$ for tuple X .



Cont'd (Issue)

- If A_k is continuous-valued,
- Gaussian distribution with a mean μ and standard deviation σ , defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i}).$$

- A zero probability cancels the effects of all the other (posteriori) probabilities (on C_i) involved in the product.
- Probability estimation is known as the Laplacian correction or Laplace estimator can be applied.



Rule-Based Classification

- Rule-based classifiers - the learned model is represented as a set of IF-THEN rules
 - E.g., a decision tree, sequential covering algorithm
- IF condition THEN conclusion.
 - R1: IF age = youth AND student = yes THEN buys computer = yes.
or
 - R1: (age = youth) $\wedge$ (student = yes) $\Rightarrow$ (buys computer = yes).
- The “IF” part (or left side) of a rule is known as the rule antecedent or precondition.
- The “THEN” part (or right side) is the rule consequent.



Cont'd

- If the condition (i.e., all the attribute tests) in a rule antecedent holds true for a given tuple, it can be said that the rule antecedent is **satisfied** (or simply, that the rule is satisfied) and that the rule **covers** the tuple.
- A rule R can be assessed by its **coverage** and **accuracy**.
- Given a tuple, X, from a class labeled data set, D
- n_{covers} be the number of tuples covered by R
- n_{correct} be the number of tuples correctly classified by R
- $|D|$ be the number of tuples in D

$$\text{coverage}(R) = \frac{n_{\text{covers}}}{|D|}$$
$$\text{accuracy}(R) = \frac{n_{\text{correct}}}{n_{\text{covers}}}.$$



Cont'd

- Rule accuracy and coverage
 - $\text{coverage}(R1) = 2/14 = 14.28\%$
 - $\text{accuracy}(R1) = 2/2 = 100\%$
- How rule-based classification can be used to predict the class label of a given tuple, X?
- If a rule is satisfied by X, the rule is said to be **triggered**.
- $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit rating} = \text{fair})$.
- If R1 is the only rule **satisfied**, then the rule **fires** by returning the class prediction for X.



Conflict resolution strategy

- Size ordering
 - the triggering rule with the most attribute tests is fired.
- Rule ordering
 - prioritizes the rules
 - class-based ordering
 - rule-based ordering
- default rule



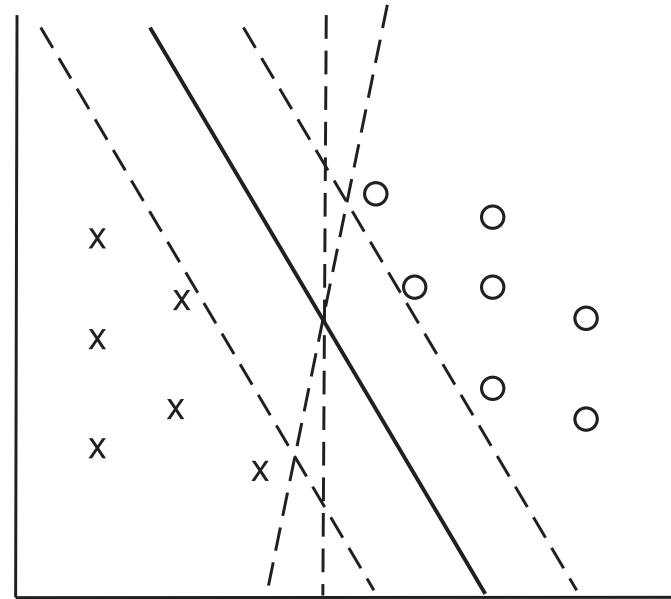
Other Classifiers

- Support Vectors Machine(SVM)
- K-Nearest Neighbors (KNN)
- Linear Regression
- Deep Learning
 - Optimizer
 - Activation
 - Learning rate
 - Batch size
 - Dropout
 - Early stop etc.,



Support Vector Machine Classifiers

- Simple 2-dimensional example:
 - Points are in two classes
 - Find a line (**maximum margin line**) s.t. line divides two classes, and distance from nearest point in either class is maximum





Support Vector Machine

- In n -dimensions points are divided by a plane, instead of a line
- SVMs can be used separators that are curve, not necessarily linear, by transforming points before classification
 - Transformation functions may be non-linear and are called kernel functions
 - Separator is a plane in the transformed space, but maps to curve in original space
- There may not be an exact planar separator for a given set of points
 - Choose plane that best separates points
- N-ary classification can be done by N binary classifications
 - In class i vs. not in class i .



Lazy vs. Eager Learning

- Lazy vs. eager learning
 - Lazy learning (e.g., instance-based learning): Simply stores training data (or only minor processing) and waits until it is given a test tuple
 - Eager learning (the above discussed methods): Given a set of training tuples, constructs a classification model before receiving new (e.g., test) data to classify
- Lazy: less time in training but more time in predicting
- Accuracy
 - Lazy method effectively uses a richer hypothesis space since it uses many local linear functions to form an implicit global approximation to the target function
 - Eager: must commit to a single hypothesis that covers the entire instance space



Lazy Learner: Instance-Based Methods

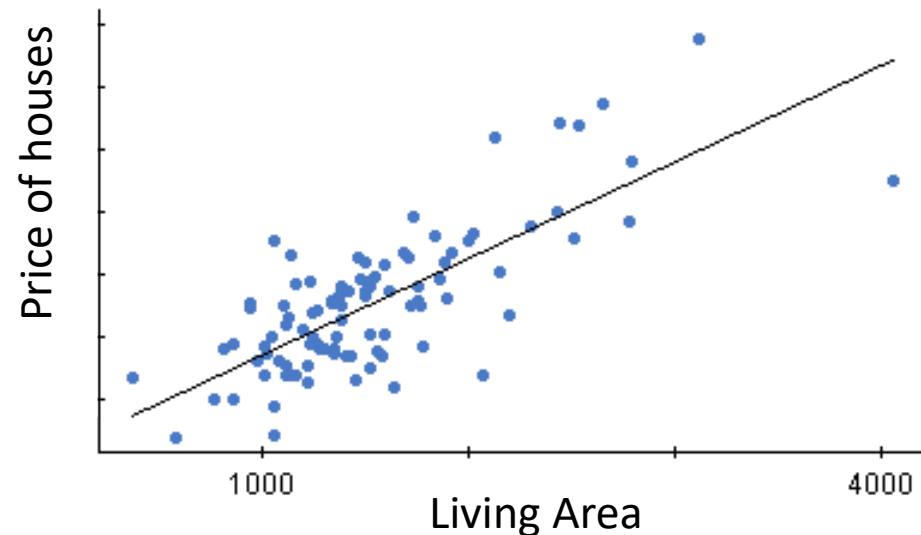
- Instance-based learning:
 - Store training examples and delay the processing (“lazy evaluation”) until a new instance must be classified
- Typical approaches
 - k-nearest neighbor approach
 - Instances represented as points in a Euclidean space.
 - Case-based reasoning
 - Uses symbolic representations and knowledge-based inference
 - Uses a database of problem solutions to solve new problems



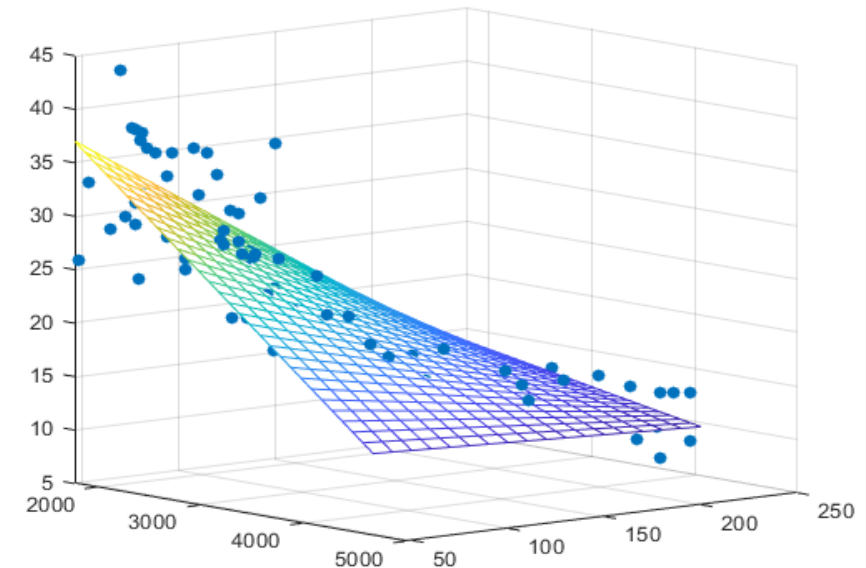
Linear Classifiers

- **Linear Regression Problem: Example**

- Mapping from independent attributes to **continuous value**: $x \Rightarrow y$
- {living area} $\Rightarrow$ Price of the house
- {college; major; GPA} $\Rightarrow$ Future Income



FIS



UCSY

...



Linear Regression Problem: Model

- Linear regression
 - Data: n independent objects
 - Observed Value: $y_i, i = 1, 2, 3, \dots, n$
 - p -dimensional attributes: $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T, i = 1, 2, 3, \dots, n$
 - Model:
 - Weight vector: $w = (w_1, w_2, \dots, w_p)$
 - $y_i = w^T x_i + b$
 - The weight vector w and bias b are the model parameter learnt by data



Linear Regression Model: Solution

➤ Least Square Method

➤ Cost / Loss Function: $L(w, b) = \sum_{i=1}^n (y_i - wx_i - b)^2$

➤ Optimization Goal: $\operatorname{argmin} L(w, b) = \sum_{i=1}^n (y_i - wx_i - b)^2$

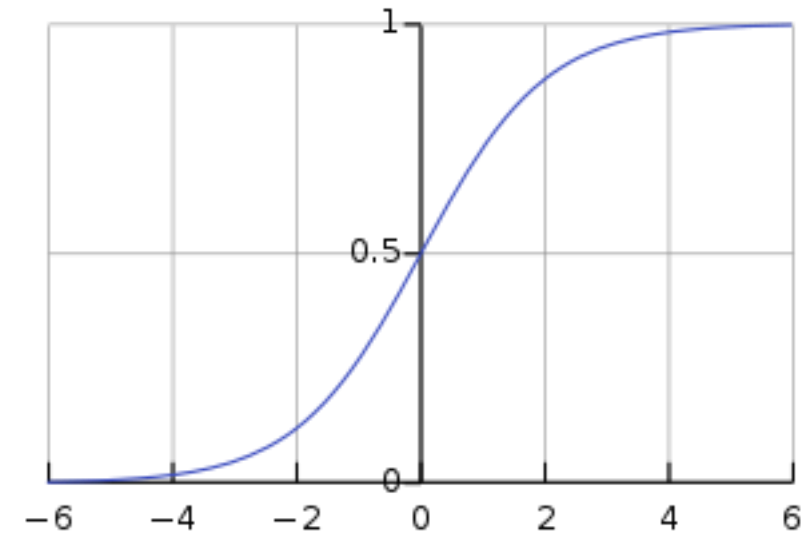
➤ Closed-form solution:

$$\begin{aligned} \text{➤ } w &= \frac{\sum_{i=1}^n x_i(y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - n(\sum_{i=1}^n x_i)^2} & b &= \frac{1}{n} \sum_{i=1}^n (y_i - wx_i) \end{aligned}$$



Logistic Regression: General Ideas

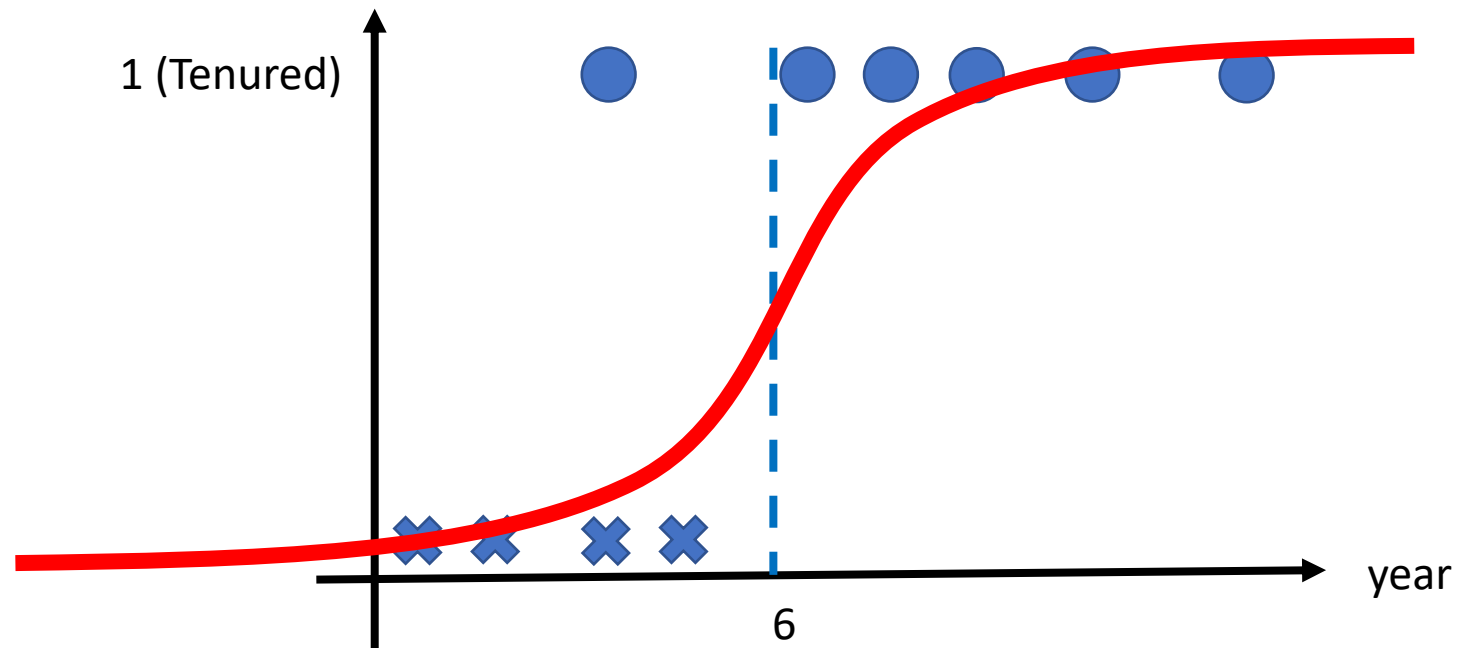
- How to solve “classification” problems by regression?
- Key idea of Logistic Regression
 - We need to transform the real value Y into a probability value $\in [0,1]$
- Sigmoid function (differentiable function) :
 - $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z+1}$
 - Projects $(-\infty, +\infty)$ to $[0, 1]$
 - Not only LR uses this function, but also neural network, deep learning
- The projected value changes sharply
 - around zero point
- Notice that $\ln \frac{y}{1-y} = w^T x + b$





Logistic Regression: An Example

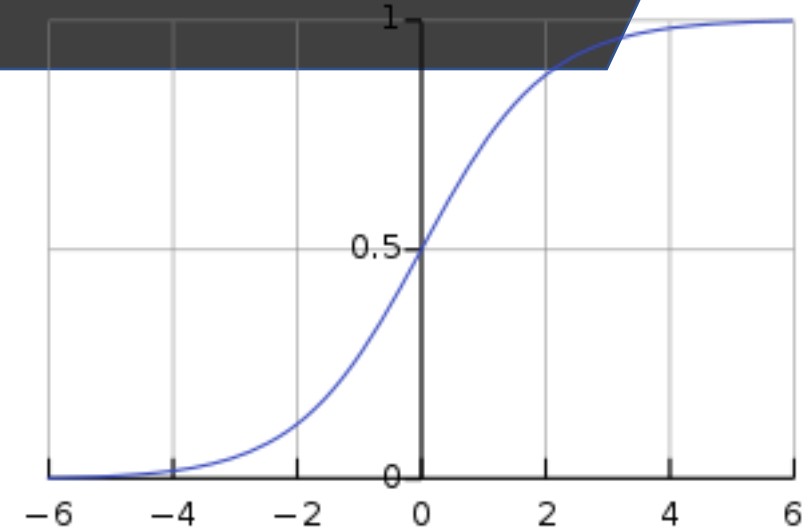
- Suppose we only consider the year as feature
 - Data points are converted by sigmoid function (“activation” function)





Logistic Regression: Model

- The prediction function to learn
- Probability that $Y=1$:
 - $p(Y = 1 | X = x; \mathbf{w}) = \text{Sigmoid}(w_0 + \sum_{i=1}^n w_i \cdot x_i)$
 - $\mathbf{w} = (w_0, w_1, w_2, \dots, w_n)$ are the parameters
- A single data object with attributes x_i and class label y_i
 - Suppose the probability of $p(\hat{y}_i = 1 | x_i, w) = p_i$, then $p(\hat{y}_i = 0 | x_i, w) = 1 - p_i$
 - $p(\hat{y}_i = y_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$
- Maximum Likelihood Estimation
 - $L = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i} = \prod_i \left(\frac{\exp(w^T x_i)}{1 + \exp(w^T x_i)} \right)^{y_i} \left(\frac{1}{1 + \exp(w^T x_i)} \right)^{1-y_i}$





Logistic Regression: Optimization

➤ Maximum Likelihood Estimation

$$\text{➤ } L = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i} = \prod_i \left(\frac{\exp(w^T x_i)}{1 + \exp(w^T x_i)} \right)^{y_i} \left(\frac{1}{1 + \exp(w^T x_i)} \right)^{1-y_i}$$

➤ Log likelihood:

$$\text{➤ } l(w) = \sum_{i=1}^N y_i \log p(Y = 1|X = x_i; \mathbf{w}) + (1 - y_i) \log(1 - p(Y = 1|X = x_i; \mathbf{w})) = \sum_{i=1}^N y_i x_i^T \mathbf{w} - \log(1 + \exp(\mathbf{w}^T x_i))$$

➤ There's no closed form solution

➤ Gradient Descent/Ascent

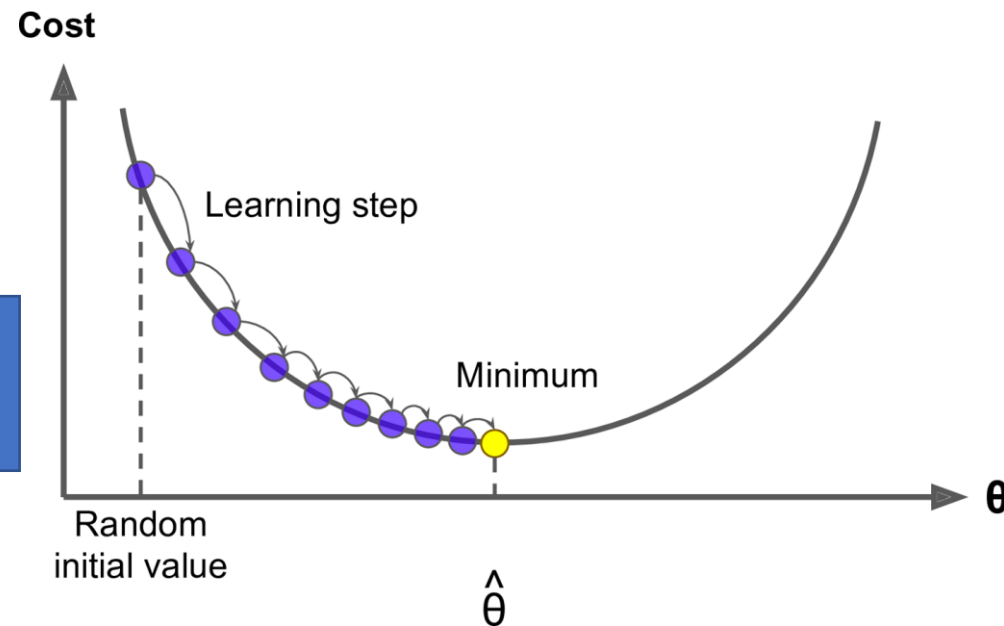


Gradient Descent

- Gradient Descent is an iterative optimization algorithm for finding the minimum of a function (e.g., the negative log likelihood)
- For a function $F(x)$ at a point $\mathbf{a}$, $F(x)$ *decreases fastest* if we go in the direction of the negative gradient c

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \gamma \nabla F(\mathbf{a}_n)$$

When the gradient is zero, we arrive at the local minimum





Gradient Descent

$$l(w) = \ln \prod_l P(Y^l | X^l, w)$$

$$= \sum_l (Y^l - 1)(w_0 + \sum_{i=1}^n w_i X_i^l) - \ln(1 + \exp(-(w_0 + \sum_{i=1}^n w_i X_i^l)))$$

$$\frac{\partial l(w)}{\partial w_i} = \sum_l X_i^l (Y^l - P(Y^l = 1 | X^l, w))$$

Iterate until change < threshold

For all i ,

$$w_i \leftarrow w_i + \eta \sum_l X_i^l (Y^l - P(Y^l = 1 | X^l, w))$$

What is the intuition?



Cont'd

Iterate until change < threshold

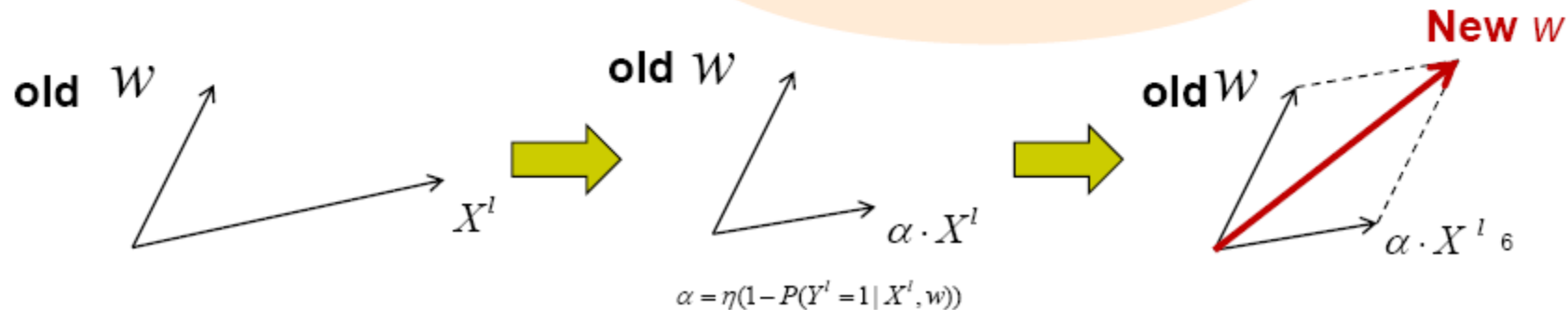
For all i ,

$$w_i \leftarrow w_i + \eta \sum_l X_i^l (Y^l - P(Y^l = 1 | X^l, w))$$

Feature, true label, current prediction

If $Y^l = 1$

$$w \leftarrow w + \eta \times X^l (1 - P(Y^l = 1 | X^l, w))$$





Cont'd

Iterate until change < threshold

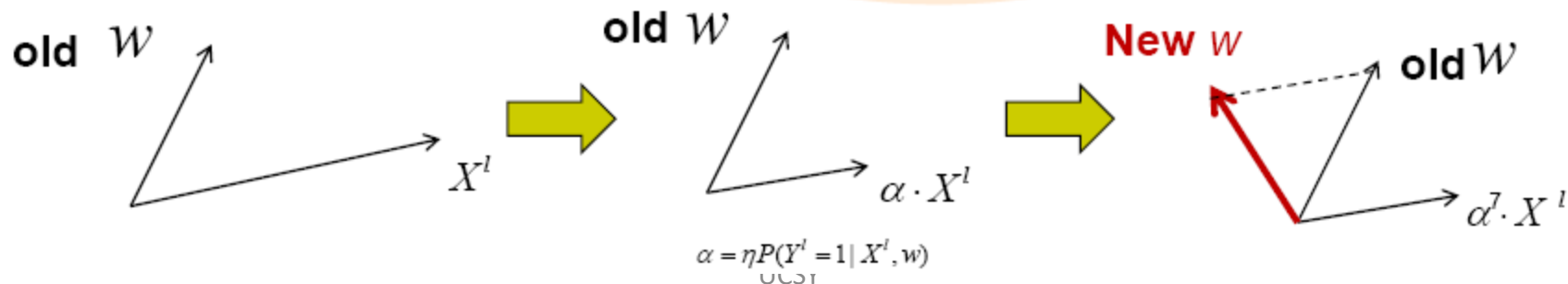
For all i ,

$$w_i \leftarrow w_i + \eta \sum_l X_i^l (Y^l - P(Y^l = 1 | X^l, w))$$

Feature, true label, current prediction

If $Y^l = 0$

$$w \leftarrow w - \eta \times X^l P(Y^l = 1 | X^l, w)$$





Model Evaluation and Selection (Cont'd)

- In addition to accuracy-based measures, classifiers can also be compared with respect to the following additional aspects:
 - **Speed**: This refers to the computational costs involved in generating and using the given classifier.
 - **Robustness**: This is the ability of the classifier to make correct predictions given noisy data or data with missing values. Robustness is typically assessed with a series of synthetic data sets representing increasing degrees of noise and missing values.
 - **Scalability**: This refers to the ability to construct the classifier efficiently given large amounts of data. Scalability is typically assessed with a series of data sets of increasing size.
 - **Interpretability**: This refers to the level of understanding and insight that is provided by the classifier or predictor. Interpretability is subjective and therefore more difficult to assess.



Model Evaluation and Selection (Cont'd)

➤ Bootstrap

- Bootstrapping tends to be overly optimistic.
- It works best with small data sets.
- The bootstrap method samples the given training tuples uniformly with replacement.
- There are several bootstrap methods. A commonly used one is the .632 bootstrap.
- The overall accuracy of the model, M , is then estimated as

$$Acc(M) = \frac{1}{k} \sum_{i=1}^k (0.632 \times Acc(M_i)_{test_set} + 0.368 \times Acc(M_i)_{train_set}),$$

- $Acc(M_i)_{test_set}$ is the accuracy of the model obtained with bootstrap sample i when it is applied to **test set i** .
- $Acc(M_i)_{train_set}$ is the accuracy of the model obtained with boot strap sample i when it is applied to the **original set of data tuples**.



Model Evaluation and Selection (Cont'd)

- Model Selection Using Statistical Tests of Significance
- Comparing Classifiers Based on Cost–Benefit and ROC Curves
- The true positives, true negatives, false positives, and false negatives are also useful in assessing the costs and benefits (or risks and gains) associated with a classification model.
- Receiver operating characteristic curves (ROC) are a useful visual tool for comparing two classification models.
- An ROC curve for a given model shows the trade-off between the true positive rate (TPR) and the false positive rate (FPR).
- To plot an ROC curve for a given classification model, M , the model must be able to return a probability of the predicted class for each test tuple.
- The diagonal shows where, for every true positive, we are equally likely to encounter a false positive.



ROC Curve

<i>Tuple #</i>	<i>Class</i>	<i>Prob.</i>	<i>TP</i>	<i>FP</i>	<i>TN</i>	<i>FN</i>	<i>TPR</i>	<i>FPR</i>
1	<i>P</i>	0.90	1	0	5	4	0.2	0
2	<i>P</i>	0.80	2	0	5	3	0.4	0
3	<i>N</i>	0.70	2	1	4	3	0.4	0.2
4	<i>P</i>	0.60	3	1	4	2	0.6	0.2
5	<i>P</i>	0.55	4	1	4	1	0.8	0.2
6	<i>N</i>	0.54	4	2	3	1	0.8	0.4
7	<i>N</i>	0.53	4	3	2	1	0.8	0.6
8	<i>N</i>	0.51	4	4	1	1	0.8	0.8
9	<i>P</i>	0.50	5	4	0	1	1.0	0.8
10	<i>N</i>	0.40	5	5	0	0	1.0	1.0



Cont'd

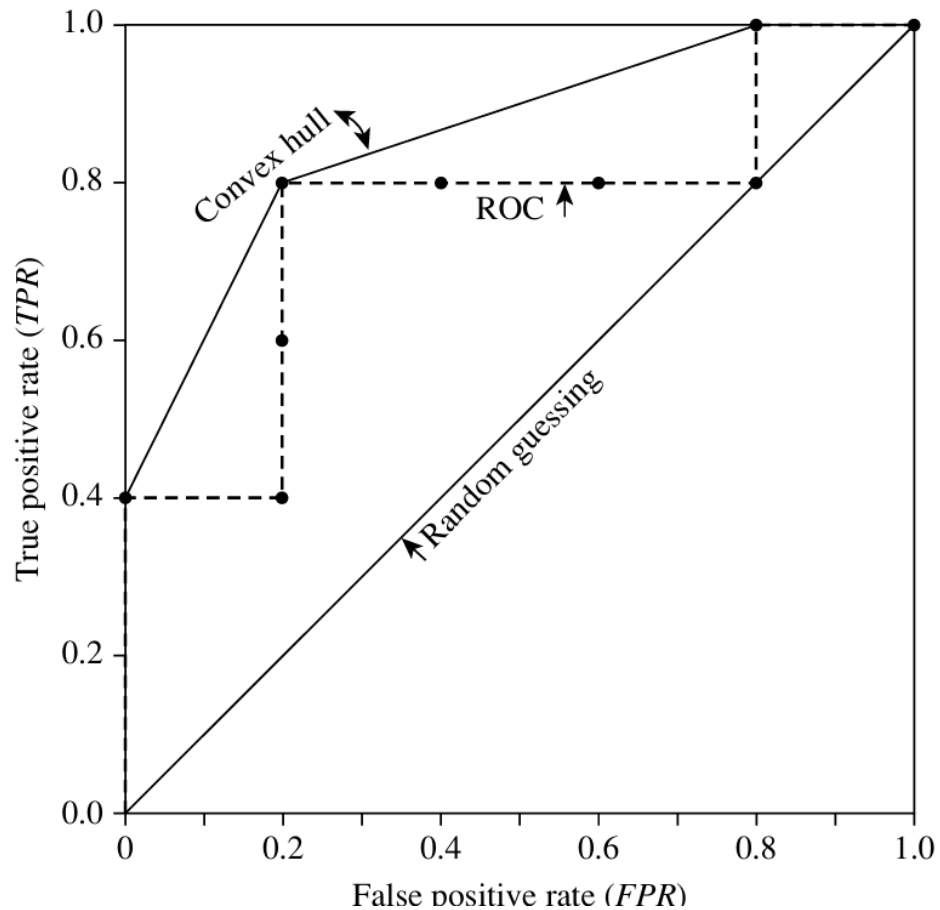


Fig: ROC curve for the data in Figure

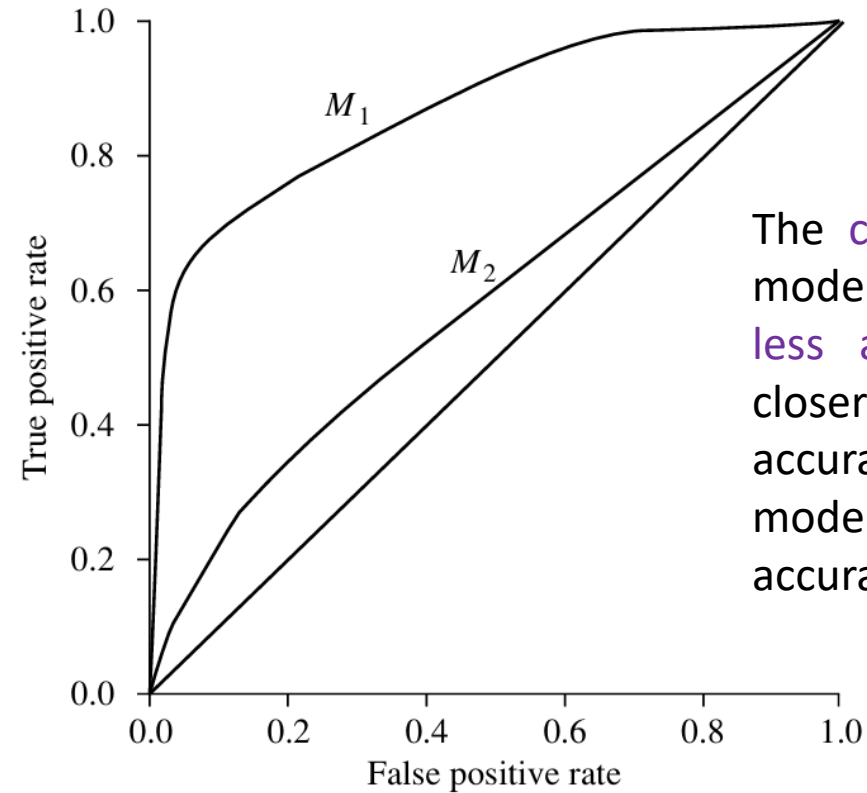
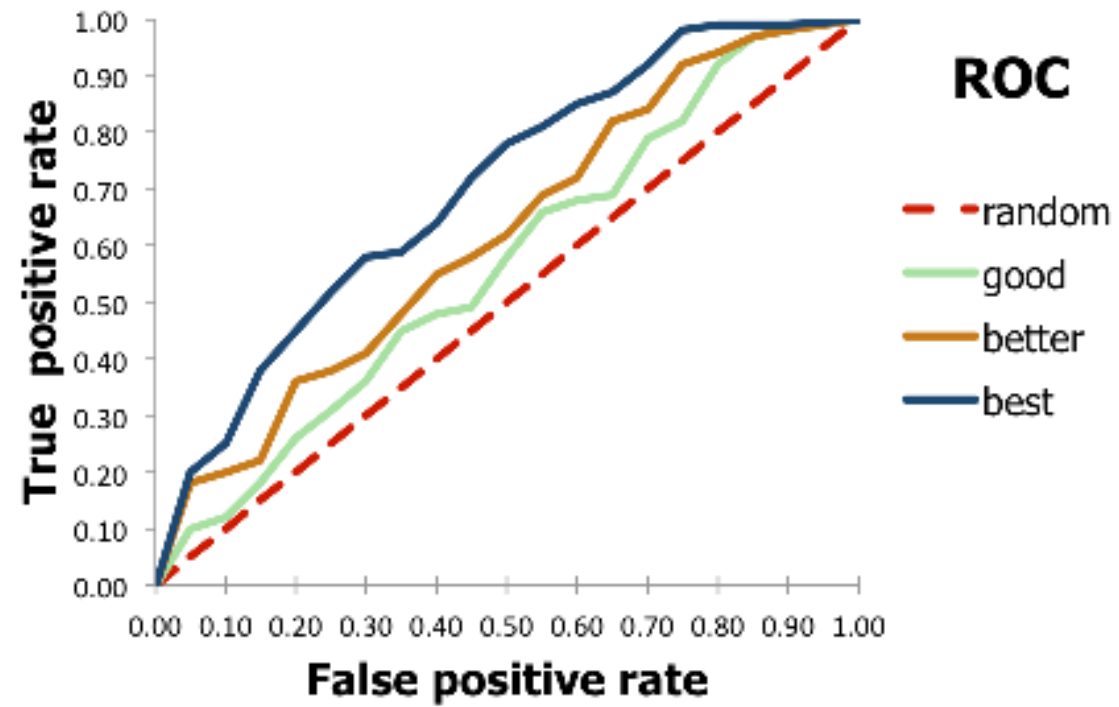


Fig: ROC curves of two classification models, M1 and M2

The **closer** the ROC curve of a model is to the diagonal line, the **less accurate** the model. The closer the area is **to 0.5**, the less accurate the corresponding model is. A model with perfect accuracy will have an area **of 1.0**.



Cont'd





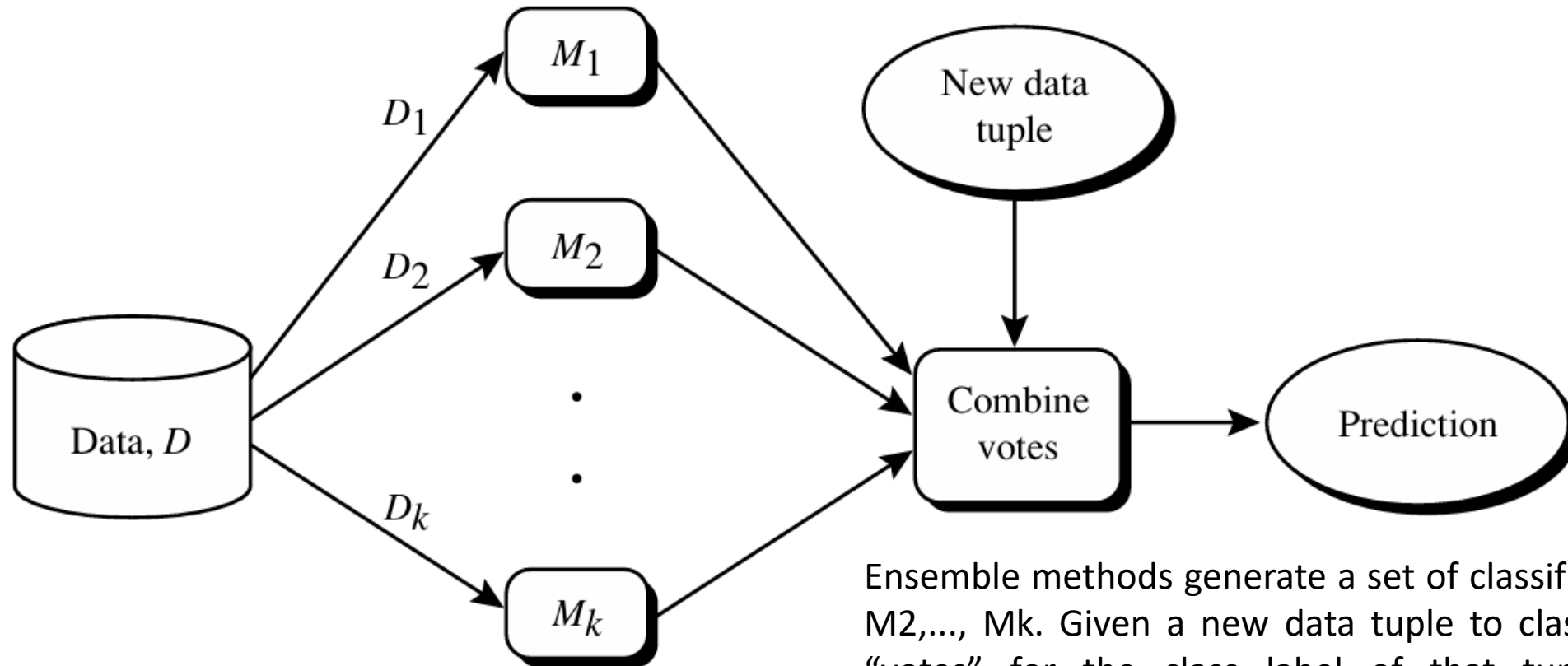
Techniques to Improve Classification Accuracy

➤ Ensemble methods

- An ensemble for classification is a **composite** model, made up of **a combination of classifiers**.
- The individual classifiers vote, and a class label prediction is returned by the ensemble based on the collection of votes.
- Ensembles tend to be more accurate than their component classifiers. Eg, an ensemble that performs **majority voting**
- Bagging, boosting, and random forests are examples of ensemble methods.



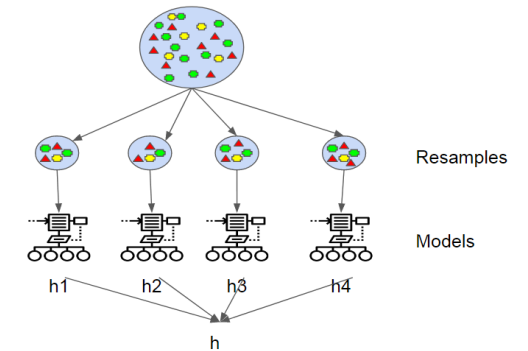
Cont'd



Ensemble methods generate a set of classification models, M_1 , $M_2, \dots, M_k$. Given a new data tuple to classify, each classifier "votes" for the class label of that tuple. The ensemble combines the votes to return a class prediction.

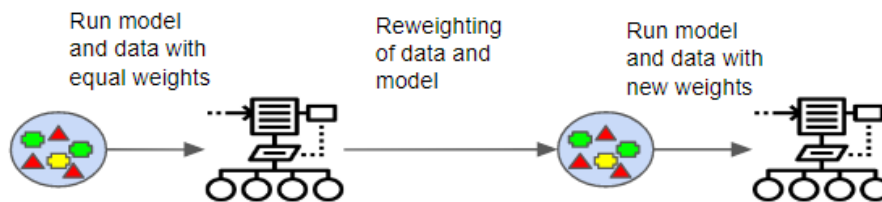
Bagging

- Given a set, D , of d tuples, bagging works as follows.
- For iteration i ($i = 1, 2, \dots, k$), a training set, D_i , of d tuples is sampled with replacement from the original set of tuples, D .
- Note that the term bagging stands for bootstrap aggregation.
- Each training set is a bootstrap sample.
- A classifier model, M_i , is learned for each training set, D_i .
- To classify an unknown tuple, X , each classifier, M_i , returns its class prediction, which counts as one vote.
- The bagged classifier, M^* , counts the votes and assigns the class with the most votes to X .
- Bagging can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple.



Boosting

- In **boosting**, **weights** are also assigned to each training tuple.
- A series of k classifiers is iteratively learned.
- After a classifier, M_i , is learned, the weights are updated to allow the subsequent classifier, M_{i+1} , to “**pay more attention**” to the training tuples that were misclassified by M_i .
- The final boosted classifier, M^* , combines the votes of each individual classifier, where the weight of each classifier’s vote is a function of its accuracy.





AdaBoost (Adaptive Boosting)

- Given data D , a data set of d class-labeled tuples, $(X_1, y_1), (X_2, y_2), \dots, (X_d, y_d)$, where y_i is the class label of tuple X_i .
- Initially, AdaBoost assigns each training tuple an equal weight of $1/d$.
- Generating k classifiers for the ensemble requires k rounds through the rest of the algorithm.
- In round i , the tuples from D are sampled to form a training set, D_i , of size d .
- Each tuple's chance of being selected is based on its weight.
- The weights of the training tuples are then adjusted according to how they were classified.
- If a tuple was incorrectly classified, its weight is increased.
- If a tuple was correctly classified, its weight is decreased.
- A tuple's weight reflects how difficult it is to classify— the higher the weight, the more often it has been misclassified.
- These weights will be used to generate the training samples for the classifier of the next round.



Cont'd

- To compute the error rate of model M_i , we sum the weights of each of the tuples in D_i that M_i misclassified. That is,
$$error(M_i) = \sum_{j=1}^d w_j \times err(X_j),$$
 - where $err(X_j)$ is the misclassification error of tuple X_j :
- If the tuple was misclassified, then $err(X_j)$ is 1; otherwise, it is 0.
- If the performance of classifier M_i is so poor that its error exceeds 0.5, then abandon it.
- Instead, try again by generating a new D_i training set, from which derive a new M_i .
- The error rate of M_i affects how the weights of the training tuples are updated.
- If a tuple in round i was correctly classified, its weight is multiplied by $error(M_i) / (1 - error(M_i))$.
- Once the weights of all the correctly classified tuples are updated, the weights for all tuples (including the misclassified ones) are normalized so that their sum remains the same as it was before.
- To normalize a weight, we multiply it by the sum of the old weights, divided by the sum of the new weights.
- As a result, the weights of mis classified tuples are increased and the weights of correctly classified tuples are decreased.



Cont'd

- Once boosting is complete, how is the ensemble of classifiers used to predict the class label of a tuple, X ?”
- Unlike bagging, where each classifier was assigned an equal vote, boosting assigns a weight to each classifier’s vote, based on how well the classifier performed.
- The lower a classifier’s error rate, the more accurate it is, and therefore, the higher its weight for voting should be. The weight of classifier M_i ’s vote is

$$\log \frac{1 - \text{error}(M_i)}{\text{error}(M_i)}.$$

- For each class, c , we sum the weights of each classifier that assigned class c to X .
- The class with the highest sum is the “winner” and is returned as the class prediction for tuple X .



Cont'd

- Because of the way boosting focuses on the misclassified tuples, it risks overfitting the resulting composite model to such data.
- Therefore, sometimes the resulting “boosted” model may be less accurate than a single model derived from the same data.
- Bagging is less susceptible to model overfitting.
- While both can significantly improve accuracy in comparison to a single model, boosting tends to achieve greater accuracy.



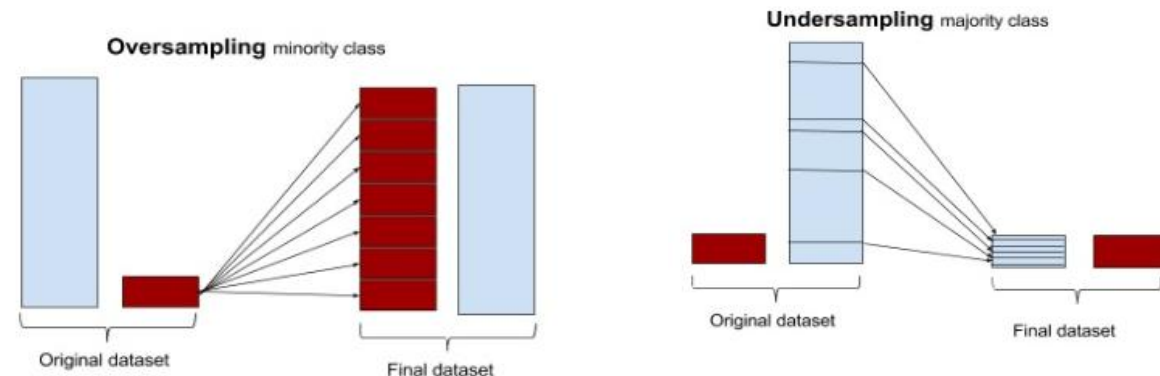
Random Forests

- Each of the classifiers in the ensemble is a decision tree classifier so that the collection of classifiers is a “forest.”
- Random forests are comparable in accuracy to AdaBoost, yet are more robust to errors and outliers.
- The generalization error for a forest converges as long as the number of trees in the forest is large.
- Thus, overfitting is not a problem.
- The accuracy of a random forest depends on the strength of the individual classifiers and a measure of the dependence between them.



Improving Classification Accuracy of Class-Imbalanced Data

- Given two-class data, the data are class-imbalanced if the main class of interest (the positive class) is represented by only a few tuples, while the majority of tuples represent the negative class.
- For multiclass-imbalanced data, the data distribution of each class differs substantially where, again, the main class or classes of interest are rare.
- The class imbalance problem is closely related to cost-sensitive learning, wherein the costs of errors, per class, are not equal.
- General approaches for improving the classification accuracy of class-imbalanced data.
 - (1) oversampling,
 - (2) under sampling,
 - (3) threshold moving, and
 - (4) ensemble techniques.





Cont'd

- General approaches with respect to the two-class imbalance data problem, where the higher-cost classes are rarer than the lower-cost classes.
- **Oversampling** works by resampling the positive tuples so that the resulting training set contains an equal number of positive and negative tuples.
- **Undersampling** works by decreasing the number of negative tuples. It randomly eliminates tuples from the majority (negative) class until there are an equal number of positive and negative tuples.
- **SMOTE**(Synthetic Minority Over-sampling Technique) algorithm uses oversampling where synthetic tuples are added, which are “close to” the given positive tuples in tuple space.
- The **threshold-moving** approach to the class imbalance problem does not involve any sampling. It applies to classifiers that, given an input tuple, return a continuous output value.



End of Lecture!

hsumyatmo@ucsy.edu.mm